

Laboratorio 141: [Reto] Ejercicio de Scripting en Python (Números Primos)

Dificultad: Avanzada / Reto Integrador

Tiempo Estimado: 40 minutos

Servicios Principales: Amazon EC2, SSH (Secure Shell), Linux Terminal

Resumen y Objetivos

A diferencia de las prácticas anteriores en un IDE gráfico (Cloud9), en este laboratorio te enfrentarás a un escenario del mundo real: conectarte de forma remota a un servidor Linux sin interfaz gráfica (headless) y desarrollar código directamente en su terminal. Al finalizar esta práctica, serás capaz de:

- Extraer las credenciales de acceso (llaves PEM/PPK) e IPs públicas desde la plataforma de laboratorio de AWS.
 - Establecer una conexión segura vía SSH a una instancia Amazon EC2.
 - Desarrollar un script en Python 3 que implemente lógica matemática compleja (identificación de números primos utilizando bucles anidados).
 - Ejecutar el script por línea de comandos y redirigir/guardar la salida en un archivo de texto plano.
-

Análisis del Escenario

Como Ingeniero de Soporte Cloud, acceder a servidores mediante SSH es una de las habilidades más críticas de tu día a día. El diagnóstico de este reto indica la necesidad de comprobar el correcto aprovisionamiento de un entorno de ejecución de Python 3 dentro de una instancia EC2 (Linux Host). El cálculo de números primos es un excelente benchmark de CPU (prueba de rendimiento estresante para el procesador). Desarrollar un script puro que interactúe con el sistema de archivos (File I/O) para guardar los resultados demuestra la capacidad de automatizar reportes dentro del sistema operativo base.

Desarrollo de las Tareas

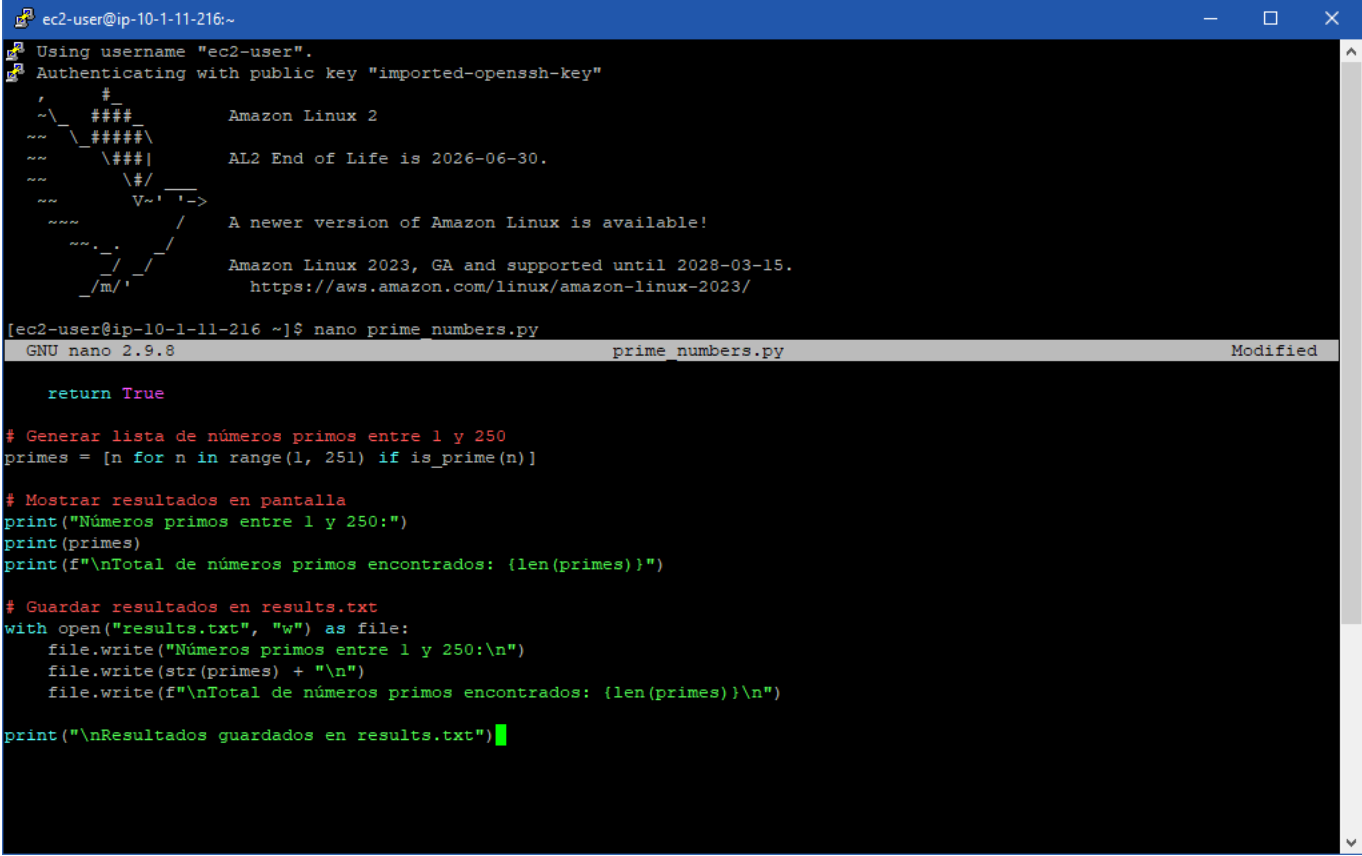
Tarea: Escribir el script de Números Primos

Debes escribir un código para identificar qué números son divisibles únicamente por 1 y por sí mismos, dentro del rango 1 al 250.

1. Ya conectado a la terminal del Linux Host, **abre** un editor de texto por línea de comandos (como **nano**) creando el archivo del script:

```
nano primes_script.py
```

2. **Escribe** el siguiente código en Python 3:



```

ec2-user@ip-10-1-11-216:~
Using username "ec2-user".
Authenticating with public key "imported-openssh-key"

  ____
 /  _ \   ____
/  / \  /  _ \   Amazon Linux 2
/_____\/_  \ /  / \   AL2 End of Life is 2026-06-30.
         \_/_ \_/_ \
         V~' '->
         A newer version of Amazon Linux is available!

         Amazon Linux 2023, GA and supported until 2028-03-15.
         https://aws.amazon.com/linux/amazon-linux-2023/

[ec2-user@ip-10-1-11-216 ~]$ nano prime_numbers.py
GNU nano 2.9.8 prime_numbers.py Modified

return True

# Generar lista de números primos entre 1 y 250
primes = [n for n in range(1, 251) if is_prime(n)]

# Mostrar resultados en pantalla
print("Números primos entre 1 y 250:")
print(primes)
print(f"\nTotal de números primos encontrados: {len(primes)}")

# Guardar resultados en results.txt
with open("results.txt", "w") as file:
    file.write("Números primos entre 1 y 250:\n")
    file.write(str(primes) + "\n")
    file.write(f"\nTotal de números primos encontrados: {len(primes)}\n")

print("\nResultados guardados en results.txt")

```

3. **Guarda** el archivo y **sal** del editor. En **nano**, presiona **Ctrl+O**, luego ENTER para confirmar, y finalmente **Ctrl+X** para salir.

Tarea 4: Ejecutar y validar los resultados

1. Para obtener la ruta absoluta de tu script (como requiere el laboratorio), **ejecuta**:

```
readlink -f primes_script.py
```

(Toma nota de esta ruta, usualmente será `/home/ec2-user/primes_script.py`).

2. **Ejecuta** el script forzando la versión 3 de Python:

```
python3 primes_script.py
```

3. **Verifica** el archivo de salida para asegurarte de que los resultados se escribieron correctamente, mostrando su contenido en pantalla:

```
cat results.txt
```

```
ec2-user@ip-10-1-11-214:~  
Primo encontrado: 241  
¡Proceso completado! Revisa el archivo results.txt  
[ec2-user@ip-10-1-11-214 ~]$ cat results.txt  
2  
3  
5  
7  
11  
13  
17  
19  
23  
29  
31  
37  
41  
43  
47  
53  
59  
61  
67  
71  
73  
79  
83  
89  
97  
101  
  
101  
103  
107  
109  
113  
127  
131  
137  
139  
149  
151  
157  
163  
167  
173  
179  
181  
191  
193  
197  
199  
211  
223  
227  
229  
233  
239  
241  
ec2-user@ip-10-1-11-214 ~]$
```

💡 Respuestas Analíticas

- **¿Cómo funciona la lógica del bloque `for ... else` en Python?** *Análisis:* Esta es una característica avanzada y peculiar de Python. En el bloque de código provisto para el reto, usamos `for i in range(2, num):`. Si el bucle iteró a través de todos los números sin encontrar ningún divisor (es decir, el bloque `if` jamás activó el comando `break`), la instrucción `else` adjunta al bucle `for` (no al `if`) se ejecuta. Es una forma extremadamente elegante de identificar un número primo sin necesitar banderas booleanas intermedias (ej. `es_primo = True`).
- **¿Por qué usamos `python3` explícitamente y no solo `python` en el Linux Host?** *Análisis:* En muchas imágenes del sistema operativo Amazon Linux (incluidas las AMI utilizadas en estos laboratorios de re/Start), `python` apunta por defecto a Python 2.7 por razones de compatibilidad con software antiguo del sistema (legacy). Invocar explícitamente `python3` asegura que el intérprete moderno gestione nuestro código, evitando fallos de sintaxis en funciones como el formateo de impresión (`print(f"...")`).

